

Parallel associative scan

a)

Up sweep:

Down sweep:

24								0							
11				13				0				11			
10	1	8	5	0	10	11	19								
4	6	1	0	1	7	3	2	0	4	10	11	11	12	19	22

b)

Naive cumulative sum algorithm:

Time complexity $O(N)$

Parallel associative scan:

Time per level: $O(1)$ (due to parallelization)

total number of layers: $\log_2 N$

$\Rightarrow$ Total time complexity: $O(2 \log N)$

c)

Redefine:

$$y_i \equiv \bar{A}_i, \quad x_i \equiv z_i$$

where $\bar{A}_i = \bar{A}$ ($\bar{A}$ constant in time)

$$a_i \equiv \bar{A}, \quad b_i = \bar{B} u_i$$

$\circ$: matrix-matrix multiplication

$\otimes$: matrix-vector multiplication

$\oplus$: vector addition

Check:

$$i = 0$$

$$y_0 = a_0 = \bar{A}, \quad x_0 = b_0 = \bar{B} u_0$$

$$i > 0;$$

$$y_i = y_{i-1} \circ a_i = \bar{A}_{i-1} \bar{A} = \bar{A}^i$$

$$x_i = x_{i-1} \circ b_i = \bar{A} z_{i-1} + \bar{B} u_i = z_i$$

Sheet 10

```
In [18]: from jax.lax import associative_scan
from jax.random import randint, PRNGKey
import jax.numpy as jnp
import jax
import time
import os
from matplotlib import pyplot as plt
```

```
In [19]: def cumsum_sequential(sequence):
    cumsum = jnp.zeros_like(sequence)
    cumsum = cumsum.at[0].set(sequence[0])
    for i in range(1, len(sequence)):
        cumsum = cumsum.at[i].set(cumsum[i - 1] + sequence[i])
    return cumsum

def cumsum_associative_scan(sequence):
    def associative_binary_operator(x, y):
        return x + y
    return associative_scan(associative_binary_operator, sequence)
```

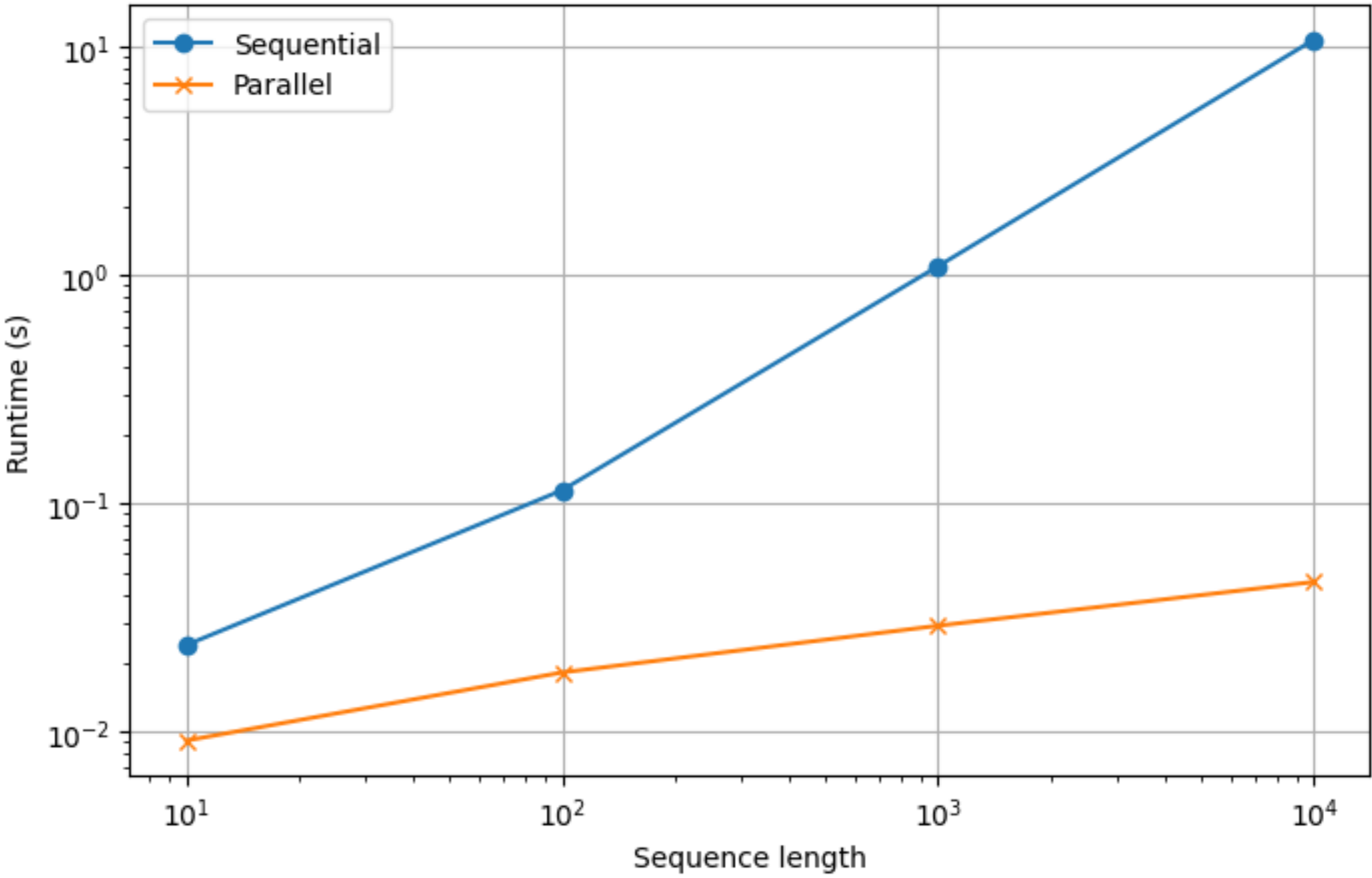
```
In [20]: lengths = [10, 100, 1000, 10000]
times_sequential = []
times_parallel = []

for length in lengths:
    sequence = randint(PRNGKey(0), (length, ), 0, 10,)

    # Sequential
    start = time.time()
    cumsum_sequential(sequence).block_until_ready()
    end = time.time()
    times_sequential.append(end - start)

    # Parallel
    start = time.time()
    cumsum_associative_scan(sequence).block_until_ready()
    end = time.time()
    times_parallel.append(end - start)
```

```
In [21]: # Plot
plt.figure(figsize=(8, 5))
plt.plot(lengths, times_sequential, label='Sequential', marker='o')
plt.plot(lengths, times_parallel, label='Parallel', marker='x')
plt.xlabel('Sequence length')
plt.ylabel('Runtime (s)')
plt.legend()
plt.grid(True)
plt.xscale('log')
plt.yscale('log')
plt.show()
```



```
In [ ]:
```